

Insertion Sort

Another commonly studied *sorting* algorithm is the *insertion sort*. **Insertion Sort** is a simple, *comparison-based* sorting algorithm that builds a sorted array (or list) *one element at a time*. It works by taking each element from the *unsorted portion* of the array (or list) and inserting it into its correct position in the *sorted portion*. The algorithm divides the array (or list) into *two parts*:

- ✚ **Sorted portion:** Initially contains the first element.
- ✚ **Unsorted portion:** Contains the remaining elements.

For each element in the *unsorted portion*, Insertion Sort compares it with the elements in the *sorted portion*, shifting larger elements to make space, and then inserts the element in its correct position.

Example: Let's sort the List [5, 2, 8, 1, 9] using *Insertion Sort*.

Step-by-Step Working

Initial Array: [5, 2, 8, 1, 9]

- ✚ Sorted portion: [5]
- ✚ Unsorted portion: [2, 8, 1, 9]

Step 1: Take 2 (next element from unsorted portion).

- ✚ Compare 2 with 5. Since $2 < 5$, shift 5 to the right.
- ✚ Insert 2 in the correct position.
- ✚ Array: [2, 5, 8, 1, 9]
- ✚ Sorted portion: [2, 5]
- ✚ Unsorted portion: [8, 1, 9]

Step 2: Take 8.

- ✚ Compare 8 with 5. Since $8 > 5$, no shifting is needed.
- ✚ Insert 8 after 5.
- ✚ Array: [2, 5, 8, 1, 9]
- ✚ Sorted portion: [2, 5, 8]
- ✚ Unsorted portion: [1, 9]

Step 3: Take 1.

- ✚ Compare 1 with 8, 5, and 2. Since 1 is smaller than all, shift 8, 5, and 2 to the right.
- ✚ Insert 1 at the beginning.
- ✚ Array: [1, 2, 5, 8, 9]
- ✚ Sorted portion: [1, 2, 5, 8]
- ✚ Unsorted portion: [9]

Step 4: Take 9.

- ✚ Compare 9 with 8. Since $9 > 8$, no shifting is needed.
- ✚ Insert 9 after 8.
- ✚ Array: [1, 2, 5, 8, 9]
- ✚ Sorted portion: [1, 2, 5, 8, 9]
- ✚ Unsorted portion: []

Final **Sorted List** in *ascending* order: [1, 2, 5, 8, 9]

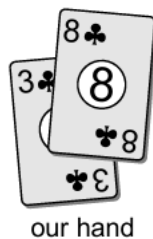
It's similar to how you might sort a hand of playing cards, placing each card in its proper place relative to the already sorted cards. Continuing with our analogy of sorting a set of playing cards to illustrate the sorting algorithms, consider five cards stacked in a deck face up:



We pick up the card from the deck and place it in our hand:



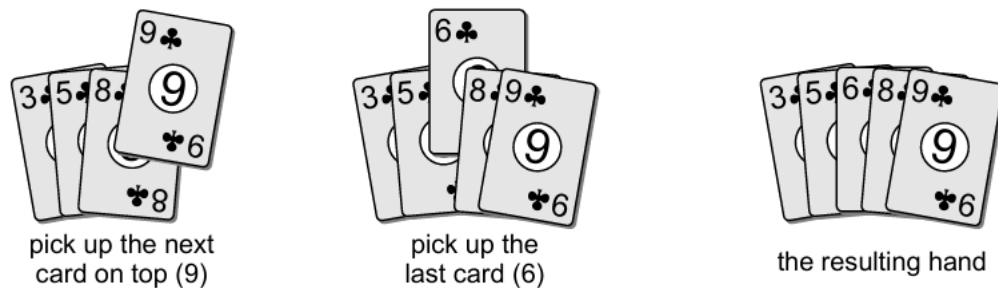
Since this is the first card, there is no decision to be made as to its position. We again pick up the top card from the deck and compare it to the card already in our hand and insert it into its proper sorted position:



After placing the 8 into our hand, the process is repeated. This time, we pick up the 5 and find its position within our hand and insert it in the proper place.



This process continues, one card at a time, until all of the cards have been removed from the table and placed in to our hand in their proper sorted position.



The insertion sort maintains a collection of sorted items and a collection of items to be sorted. In the playing card analogy, the deck represents the collection to be sorted and the cards in our hand represents those already sorted. When implementing insertion sort in a program, the algorithm maintains both the sorted and unsorted collections within the same sequence structure. The algorithm keeps the list of sorted values at the front of the sequence and picks the next unsorted value from the first of those yet to be positioned. To position the next item, the correct spot within the sequence of sorted values is found by performing a search. After finding the proper position, the slot has to be opened by shifting the items down one position.

Listing 5.7 Implementation of the insertion sort algorithm.

```
1  # Sorts a sequence in ascending order using the insertion sort algorithm.
2  def insertionSort( theSeq ):
3      n = len( theSeq )
4      # Starts with the first item as the only sorted entry.
5      for i in range( 1, n ) :
6          # Save the value to be positioned.
7          value = theSeq[i]
8          # Find the position where value fits in the ordered part of the list.
9          pos = i
10         while pos > 0 and value < theSeq[pos - 1] :
11             # Shift the items to the right during the search.
12             theSeq[pos] = theSeq[pos - 1]
13             pos -= 1
14
15         # Put the saved value into the open slot.
16         theSeq[pos] = value
```

The **insertionSort()** function starts by assuming the *first item* is in its proper position. Next, an iteration is performed over the remaining items so each value can be inserted into its proper position within the sorted portion of the sequence. The *ordered part* of the sequence is *first*, and the *uninserted part is last*. The *i* loop index variable marks the separation point between the two parts. The *inner loop* is used to find the *insertion point* within the sorted sequence and, at the same time, shift the items down to make room for the *next item*. Thus, the inner loop starts from the end of the sorted subsequence and works its way to the front. After finding the proper position, the item is inserted.

1- Where do we apply Insertion Sort?

Insertion Sort is mostly used in:

- ✚ **Small Data Sets:** It performs well for small arrays (e.g., fewer than 50 elements) due to its simplicity and low overhead.
- ✚ **Partially Sorted Arrays:** It's efficient when the array is already partially sorted, as it requires fewer comparisons and shifts.
- ✚ **Online Sorting:** When data arrives incrementally (e.g., streaming data), Insertion Sort can insert new elements into an already sorted list efficiently.
- ✚ **Adaptive Algorithms:** It's used in hybrid algorithms like Timsort (used in Python and Java) for small subarrays because of its efficiency on nearly sorted data.
- ✚ **Educational Purposes:** It's taught in computer science courses due to its simplicity and intuitive nature.

2- Advantages of Insertion Sort

- ✚ **Simple Implementation:** Easy to understand and implement, making it ideal for teaching and small-scale use.
- ✚ **Efficient for Small or Nearly Sorted Data:** Performs well ($O(n)$ in the best case) when the input is small or partially sorted.
- ✚ **In-Place Sorting:** Requires only $O(1)$ extra space, as it sorts the array in-place.
- ✚ **Stable Sorting:** Maintains the relative order of equal elements.
- ✚ **Adaptive:** Takes advantage of existing order in the input, requiring fewer operations for partially sorted arrays.
- ✚ **Online Algorithm:** Can sort data as it arrives, without needing the entire dataset upfront.

3- Disadvantages of Insertion Sort

- ✚ **Poor Performance for Large Data Sets:** Has a worst-case and average-case time complexity of $O(n^2)$, making it inefficient for large arrays compared to algorithms like QuickSort or MergeSort.
- ✚ **Not Suitable for Large, Random Data:** Requires many comparisons and shifts for unsorted or reverse-sorted arrays.
- ✚ **Not Parallelizable:** Unlike some advanced algorithms, Insertion Sort is inherently sequential, limiting its use in parallel computing environments.

4- Implementation of Insertion Sort

1) First Step is to Make *Function of Insertion Sort*.

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and key < arr[j]:  
            arr[j + 1] = arr[j] # Shift elements to the right  
            j -= 1  
        arr[j + 1] = key # Insert the key at the correct position
```

2) Call *Insertion Sort Function* for Data

```
arr = [12, 11, 13, 5, 6]
print("un-Sorted array is:", arr)
insertion_sort(arr)
print("Sorted array is:", arr)
```

3) Call *Insertion Sort Function* to *Sort Stack or Queue*

```
def insertion_sort(self):
    n = len(self.stack)
    for i in range(1, n):
        key = self.stack[i]
        j = i - 1
        while j >= 0 and key < self.stack[j]:
            self.stack[j + 1] = self.stack[j]
            j -= 1
        self.stack[j + 1] = key
        print(f"After pass {i}: {self.stack}")
    print("Sorted stack is:", self.stack)

stack = Stack()
stack.insertion_sort()
```

Hint: For the *sorting* of the *queue*, just *call* the existing *queue* instead of the *stack*.

5- Practice Questions

Below are the **3 practice questions** based on the **Insertion Sort** implementation that will help you to learn and practice **Insertion Sort** effectively with respect to Stack and Queue as well:

1. Sorting Emergency Cases in a Hospital

A **hospital** maintains a **queue of patient** criticality levels (**1 = most critical, 5 = least critical**), such as: **[4, 2, 5, 1, 3]** (**enqueued in order of arrival**).

- ✚ **Dequeue** all patients into a list.
- ✚ Use **Insertion Sort** to arrange patients in ascending order of **criticality (most critical first)**.
- ✚ **Enqueue** the sorted list back into the queue.
- ✚ Print the queue contents by **dequeuing** all elements at the end.

2. Sorting Exam Scripts by Roll Number

A **teacher** collects **student exam scripts** in a **stack** in the order they were submitted (e.g. roll numbers **[105, 101, 110, 102, 108]**).

- ✚ **Pop** all roll numbers into a list.
- ✚ Apply **Insertion Sort** to sort roll numbers in **Descending and Ascending order**.
- ✚ **Push** the sorted roll numbers back into a new stack.
- ✚ Finally, **pop all scripts** from the stack and display them.

3. Library Book Management System

A university library wants to automate the management of its books. The librarian handles **two types of book records** using different data structures:

- ✚ **Returned Books (Queue):** Books that students return are stored in the order they arrive.
- ✚ **Issued Books (Stack):** Books that students borrow are stored with the last issued book on top.

The librarian also wants to **sort** all **book IDs** when reorganizing the system.

Step 1: Build the Data Structures

- ✚ Implement a **Queue** class with the following functions:
 - ✚ **enqueue (book_id)** → add book at the rear.
 - ✚ **dequeue ()** → remove book from the front.
 - ✚ **peek ()** → get the first book without removing it.
 - ✚ **is_empty ()** and **is_full ()** → check status.
 - ✚ **display ()** → print the current queue.

- ✚ Implement a **Stack** class with the following functions:
 - ✚ `push(book_id)` → add book on top.
 - ✚ `pop()` → remove book from the top.
 - ✚ `peek()` → get the top book without removing it.
 - ✚ `is_empty()` and `is_full()` → check status.
 - ✚ `display()` → print the current stack.

Step 2: Perform Operations

- ✚ **Dequeue** all returned books into a list.
- ✚ **Pop** all issued books into another list.
- ✚ **Merge** both lists into a single list.
- ✚ Apply **Insertion Sort** to arrange all book IDs in ascending order.
- ✚ **Reinsert** the sorted IDs:
 - ✚ First half back into the **Queue (returned section)**.
 - ✚ Second half back into the **Stack (issued section)**.

Step 3: Output Requirements

- ✚ Print the **initial state** of both Queue and Stack.
- ✚ Show the **merged list** before sorting.
- ✚ Show the **sorted list** after applying **Insertion Sort**.
- ✚ Print the **reinserted Queue and Stack**.
- ✚ Finally, display all books again by:
 - ✚ **Dequeuing** from the Queue.
 - ✚ **Popping** from the Stack.